

## **Praktikum 1**

**PEKAN 1: Pengenalan Big Data & Lingkungan Kerja, serta Pengolahan Dataset  
Big Data Menggunakan Python**



**DISUSUN OLEH**

**Abdullah Al Ramadhani (2411533016)**

**DOSEN PENGAMPU**

**Afdal Dinilhak, M.Kom**

**FAKULTAS TEKNOLOGI INFORMASI**

**DEPARTEMEN INFORMATIKA**

**UNIVERSITAS ANDALAS**

**2026**

## **A. Pendahuluan**

Perkembangan volume informasi saat ini menghadirkan tantangan komputasi yang dikenal sebagai Big Data, di mana ukuran, kecepatan, dan keberagaman data udah melewati batas kemampuan pengolahan konvensional di satu mesin. Masalah di Big Data gak cuma soal jumlah data yang banyak, tapi lebih ke momen operasional di mana kapasitas memori udah gak cukup, proses pembacaan jadi sangat lambat, dan butuh strategi khusus buat ngejekusinya. Karakteristik ini direpresentasiin lewat dimensi kayak Volume, Velocity, Variety, Veracity, dan Value (5V), yang maksa praktisi data buat mulai ngukur batasan sumber daya secara objektif dan beralih dari pendekatan pengolahan biasa ke teknik efisiensi komputasi.

Buat ngerti batasan itu, butuh simulasi pengolahan langsung pakai lingkungan kerja berbasis cloud kayak Google Colab yang nyediain sumber daya terisolasi tanpa perlu instalasi software lokal. Lewat penggunaan pustaka pemrosesan data kayak pandas dan PySpark terhadap dataset NYC TLC Yellow Taxi Trip Records berskala jutaan baris, analisis difokusin pada penerapan metode-metode hemat memori. Pendekatan learning by doing ini gak cuma buktiin secara nyata gimana limitasi memori terjadi lewat pengukuran Resident Set Size (RSS), tapi juga jadi landasan awal buat mempraktikin optimasi tipe data, pemrosesan bertahap (chunking), serta perbandingan efisiensi antara format baris (CSV) dan format kolom (Parquet).

## **B. Tujuan**

Tujuan dari praktikum ini adalah:

1. Mahasiswa dapat menyiapkan notebook Colab yang siap pakai dan mendokumentasikan spesifikasi lingkungan kerjanya (Python version, library version, RAM, disk).
2. Mahasiswa dapat mengunduh dataset publik secara terprogram dan memeriksa integritas awalnya (ukuran file, jumlah baris, skema kolom).
3. Mahasiswa dapat mengukur pemakaian memori sebuah DataFrame dan menurunkannya melalui pemilihan kolom serta optimasi tipe data.
4. Mahasiswa dapat memproses file yang lebih besar dari RAM yang tersedia menggunakan teknik chunking.
5. Mahasiswa dapat menghasilkan tabel agregasi (per jam, per metode pembayaran) dan menyimpannya dalam format CSV serta Parquet.
6. Mahasiswa dapat menjalankan kueri agregasi menggunakan PySpark local mode dan menjelaskan perbedaan konseptualnya dengan pandas.

## C. Isi

### a. Langkah-langkah

#### 1. J-1. Verifikasi lingkungan kerja.

```
import sys, platform, multiprocessing

print("Python :", sys.version.split()[0])
print("OS      :", platform.platform())
print("CPU core:", multiprocessing.cpu_count())

for nama in ["pandas", "numpy", "pyarrow", "matplotlib"]:
    try:
        mod = __import__(nama)
        print(f"{nama} : {mod.__version__}")
    except ImportError:
        print(f"{nama} : BELUM TERPASANG")

Python : 3.13.15
OS      : Linux-6.6.122+-x86_64-with-glibc2.35
CPU core: 2
pandas : 2.2.3
numpy  : 2.1.3
pyarrow : 18.1.0
matplotlib : 3.10.0
```

Code diatas untuk memindai dan mencatat spesifikasi lingkungan eksekusi di Google Colab. Ia mengidentifikasi versi interpreter Python, platform sistem operasi, jumlah inti prosesor yang tersedia, serta versi paket kunci seperti pandas, numpy, pyarrow, dan matplotlib. Selain itu, skrip menjalankan perintah shell untuk melaporkan sisa memori RAM, ruang disk yang terpakai, dan ketersediaan runtime Java.

#### 2. J-2. Menghubungkan Google Drive dan menyiapkan struktur folder

```
from google.colab import drive
drive.mount('/content/drive')

import os
DIR_KERJA = "/content/data"
DIR_SIMPAN = "/content/drive/MyDrive/data"
os.makedirs(DIR_KERJA, exist_ok=True)
os.makedirs(DIR_SIMPAN, exist_ok=True)
print(os.listdir(DIR_SIMPAN))

Mounted at /content/drive
[]
```

Bagian ini mengaitkan akun Google Drive dengan sesi Colab yang aktif. Kemudian skrip membuat dua folder lewat modul os: satu folder kerja lokal di mesin Colab untuk operasi I/O berkecepatan tinggi, dan satu folder arsip di Google Drive untuk penyimpanan jangka panjang. Setelah itu, daftar file yang sudah berada di folder arsip ditampilkan.

### 3. J-3. Menyiapkan alat ukur waktu dan memori

```
import time, os, psutil
proses = psutil.Process(os.getpid())
catatan = []

def rss_mb():
    return proses.memory_info().rss / 1024**2

def ukur(label, fungsi):
    """Menjalankan fungsi tanpa argumen, mencatat durasi dan pertumbuhan memori."""
    m0, t0 = rss_mb(), time.perf_counter()
    hasil = fungsi()
    detik = time.perf_counter() - t0
    delta = rss_mb() - m0
    catatan.append({"langkah": label, "detik": round(detik, 2),
                    "delta_rss_mb": round(delta, 1)})
    print(f"[{label}] {detik:.2f} s | RSS {delta:+.1f} MB")
    return hasil
```

Code diatas didefinisikan utilitas untuk mengukur performa eksekusi dari sisi waktu dan pemakaian memori. Fungsi rss\_mb mengambil nilai memori fisik aktual yang dipakai proses Python. Fungsi ukur menerima callable sebagai argumen, mencatat lama eksekusi dan selisih penggunaan memori, menyimpan catatan ke dalam daftar log, lalu mencetak ringkasannya.

### 4. J-4. Mengunduh dataset dan inspeksi awa

```
import urllib.request

URL = ("https://d37ci6vzurnychx.cloudfront.net/trip-data/"
      "yellow_tripdata_2021-01.parquet")
PATH = os.path.join(DIR_KERJA, "yellow_tripdata_2021-01.parquet")

if not os.path.exists(PATH):
    ukur("unduh dataset", lambda: urllib.request.urlretrieve(URL, PATH))

print("Ukuran file:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")

[unduh dataset] 0.61 s | RSS +1.1 MB
Ukuran file: 20.7 MB

import pyarrow.parquet as pq

meta = pq.ParquetFile(PATH).metadata
print("Jumlah baris :", f"{meta.num_rows:,}")
print("Jumlah kolom :", meta.num_columns)
print("Row group      :", meta.num_row_groups)
print("Nama kolom     :", pq.ParquetFile(PATH).schema.names)

Jumlah baris : 1,369,769
Jumlah kolom : 19
Row group    : 1
Nama kolom   : ['VendorID', 'tpep_pickup_datetime', 'tpep_dropoff_datetime',
```

Code diatas ini mengunduh berkas dataset taksi NYC TLC ke folder kerja apabila belum tersedia, sambil mencatat waktu dan memori yang digunakan selama pengunduhan. Setelah selesai, ukuran file diperiksa dan pyarrow dipakai untuk membaca metadata Parquet, termasuk jumlah kolom, row group, dan nama kolom, tanpa memuat seluruh isi data ke memori.

## 5. J-5. Memuat data dan mengukur biaya memorinya

```
import pandas as pd

KOLOM = ["tpep_pickup_datetime", "tpep_dropoff_datetime",
         "passenger_count", "trip_distance", "payment_type",
         "fare_amount", "tip_amount", "total_amount"]

penuh = ukur("baca semua kolom", lambda: pd.read_parquet(PATH))
print("Dimensi:", penuh.shape)
print("Memori :", round(penuh.memory_usage(deep=True).sum() / 1024**2, 1), "MB")

del penuh # bebaskan memori sebelum percobaan kedua
import gc; gc.collect()

df = ukur("baca kolom terpilih", lambda: pd.read_parquet(PATH, columns=KOLOM))
print("Dimensi:", df.shape)
print("Memori :", round(df.memory_usage(deep=True).sum() / 1024**2, 1), "MB")
df.head()
```

[baca semua kolom] 0.84 s | RSS +416.0 MB  
Dimensi: (1369769, 19)  
Memori : 251.0 MB  
[baca kolom terpilih] 0.22 s | RSS +78.2 MB  
Dimensi: (1369769, 8)  
Memori : 83.6 MB

|   | tpep_pickup_datetime | tpep_dropoff_datetime | passenger_count | trip_distance | payment_type | fare_amount | tip_amount | total_amount |
|---|----------------------|-----------------------|-----------------|---------------|--------------|-------------|------------|--------------|
| 0 | 2021-01-01 00:30:10  | 2021-01-01 00:36:12   | 1.0             | 2.10          | 2            | 8.0         | 0.00       | 11.80        |
| 1 | 2021-01-01 00:51:20  | 2021-01-01 00:52:19   | 1.0             | 0.20          | 2            | 3.0         | 0.00       | 4.30         |
| 2 | 2021-01-01 00:43:30  | 2021-01-01 01:11:06   | 1.0             | 14.70         | 1            | 42.0        | 8.65       | 51.95        |
| 3 | 2021-01-01 00:15:48  | 2021-01-01 00:31:01   | 0.0             | 10.60         | 1            | 29.0        | 6.05       | 36.35        |
| 4 | 2021-01-01 00:31:49  | 2021-01-01 00:48:21   | 1.0             | 4.94          | 1            | 16.5        | 4.06       | 24.36        |

```
df.info(memory_usage="deep")
df.memory_usage(deep=True).sort_values(ascending=False) / 1024**2

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1369769 entries, 0 to 1369768
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   tpep_pickup_datetime  1369769 non-null  datetime64[us]
1   tpep_dropoff_datetime 1369769 non-null  datetime64[us]
2   passenger_count        1271417 non-null  float64
3   trip_distance          1369769 non-null  float64
4   payment_type           1369769 non-null  int64
5   fare_amount            1369769 non-null  float64
6   tip_amount             1369769 non-null  float64
7   total_amount           1369769 non-null  float64
dtypes: datetime64[us](2), float64(5), int64(1)
memory usage: 83.6 MB

0
tpep_pickup_datetime    10.450508
payment_type             10.450508
tpep_dropoff_datetime    10.450508
passenger_count          10.450508
trip_distance            10.450508
tip_amount               10.450508
fare_amount              10.450508
total_amount             10.450508
Index                    0.000126
dtype: float64
```

```

sebelum = df.memory_usage(deep=True).sum() / 1024**2

df["passenger_count"] = pd.to_numeric(df["passenger_count"], downcast="float")
df["payment_type"] = df["payment_type"].astype("int8").astype("category")
for kol in ["trip_distance", "fare_amount", "tip_amount", "total_amount"]:
    df[kol] = pd.to_numeric(df[kol], downcast="float")

sesudah = df.memory_usage(deep=True).sum() / 1024**2
print(f"sebelum: {sebelum:.1f} MB -> {sesudah:.1f} MB (hemat {(100*(1-sesudah/sebelum):.1f})%")
df.dtypes

53.6 MB -> 53.6 MB (hemat 0.0%)

0

tpep_pickup_datetime    datetime64[us]
tpep_dropoff_datetime   datetime64[us]
passenger_count         float32
trip_distance           float64
payment_type            category
fare_amount             float32
tip_amount              float32
total_amount            float32

dtype: object

```

Bagian ini membandingkan beban memori saat memuat semua kolom versus hanya kolom tertentu, untuk menunjukkan efisiensi Parquet. Variabel awal dihapus dengan `del` dan `gc.collect()`, lalu dihitung penggunaan memori per kolom. Selanjutnya dilakukan optimasi tipe data melalui downcasting numerik dan konversi ke category, diikuti perhitungan persentase penghematan memori total.

## 6. J-6. Pemeriksaan kualitas data dan pembersihan (veracity)

```

df["durasi_menit"] = (df["tpep_dropoff_datetime"] -
df["tpep_pickup_datetime"]).dt.total_seconds() / 60

print(df[["trip_distance", "durasi_menit", "total_amount"]].describe())
print("\nNilai kosong per kolom:\n", df.isna().sum())

anomali = {
    "jarak <= 0" : (df["trip_distance"] <= 0).sum(),
    "durasi <= 0" : (df["durasi_menit"] <= 0).sum(),
    "durasi > 180 menit" : (df["durasi_menit"] > 180).sum(),
    "total_amount <= 0" : (df["total_amount"] <= 0).sum(),
}

for k, v in anomali.items():
    print(f"{k:22s}: {v:,} baris ({100*v/len(df):.2f}%)")

count    trip_distance    durasi_menit    total_amount
mean    1.369769e+06    1.369769e+06    1.369769e+06
std      4.631983e+00    1.391168e+01    1.747441e+01
std      3.939037e+02    1.312006e+02    1.467865e+01
min      0.000000e+00    -1.350046e+05    -4.928000e+02
25%      1.000000e+00    5.566667e+00    1.088000e+01
50%      1.700000e+00    9.066667e+00    1.388000e+01
75%      3.020000e+00    1.461667e+01    1.912000e+01
max      2.631633e+05    2.881770e+04    7.661280e+03

Nilai kosong per kolom:
tpep_pickup_datetime    0
tpep_dropoff_datetime   0
passenger_count         98352
trip_distance           0
payment_type            0
fare_amount             0
tip_amount              0
total_amount            0
durasi_menit            0
dtype: int64
jarak <= 0 : 19,952 baris (1.46%)
durasi <= 0 : 6,557 baris (0.48%)
durasi > 180 menit : 2,506 baris (0.18%)
total_amount <= 0 : 7,114 baris (0.52%)

```

```

layak = (
    (df["trip_distance"] > 0) & (df["trip_distance"] < 100) &
    df["durasi_menit"].between(1, 180) &
    (df["total_amount"] > 0)
)
bersih = df.loc[layak].copy()
print(f"{len(df):,} baris -> {len(bersih):,} baris (dibuang {len(df)-len(bersih):,})")

1,369,769 baris -> 1,329,559 baris (dibuang 40,210)

```

Code diatas melakukan inspeksi kualitas data dan pembersihan. Pertama, durasi perjalanan dalam menit dihitung dari selisih waktu turun dan waktu jemput, lalu

ditampilkan statistik deskriptif dan jumlah missing value per kolom. Setelah itu, baris anomali diidentifikasi, seperti jarak atau durasi nol atau negatif, durasi lebih dari 180 menit, dan tarif nol atau negatif. Data yang lolos filter disimpan di DataFrame baru bernama bersih, dan jumlah baris yang dibuang dilaporkan.

## 7. J-7. Memproses file lebih besar dari memori dengan chunking

```
CSV = os.path.join(DIR_KERJA, "trips.csv")
ukur("tulis CSV", lambda: bersih.to_csv(CSV, index=False))

print("Parquet:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")
print("CSV : ", round(os.path.getsize(CSV) / 1024**2, 1), "MB")

[tulis CSV] 16.65 s | RSS +5.0 MB
Parquet: 20.7 MB
CSV : 99.5 MB

def agregasi_bertahap(path, ukuran_chunk=500_000):
    jumlah_baris = 0
    total_nilai = 0.0
    per_jam = {}
    potongan = pd.read_csv(
        path,
        usecols=["tpep_pickup_datetime", "total_amount"],
        parse_dates=["tpep_pickup_datetime"],
        chunksize=ukuran_chunk)
    for chunk in potongan:
        jumlah_baris += len(chunk)
        total_nilai += chunk["total_amount"].sum()
        for jam, sub in chunk.groupby(chunk["tpep_pickup_datetime"].dt.hour):
            n, nilai = per_jam.get(jam, (0, 0.0))
            per_jam[jam] = (n + len(sub), nilai + sub["total_amount"].sum())
    return jumlah_baris, total_nilai, per_jam

baris, total, per_jam_chunk = ukur("agregasi chunking", lambda: agregasi_bertahap(CSV))
print(f'{baris:,} baris | rata-rata total_amount = {total/baris:.2f}')

[agregasi chunking] 3.12 s | RSS +10.6 MB
1,329,559 baris | rata-rata total_amount = 17.49
```

Code diatas mensimulasikan pemrosesan out of core untuk skenario file yang lebih besar dari memori. Data bersih diekspor ke CSV agar ukurannya bisa dibandingkan dengan Parquet yang lebih terkompresi. Fungsi agregasi\_bertahap membaca CSV secara bertahap sebanyak 500 ribu baris per iterasi, menghitung total transaksi dan agregasi per jam pada setiap potongan, tanpa memuat seluruh file sekaligus.

## 8. J-8. Pendekatan terdistribusi: PySpark local mode

```
!pip install -q pyspark==3.5.1
!java -version

[0.001s][warning][os,container] Cgroup memory controller path at '/sys/fs/cgroup' seems to have moved to '/../../jupyter-children', detected limits won't be accurate
[0.001s][warning][os,container] Cgroup cpu controller path at '/sys/fs/cgroup' seems to have moved to '/../../jupyter-children', detected limits won't be accurate
pyspark version "21.0.11" 2026-04-21
OpenJDK Runtime Environment (build 21.0.11+10-1-22.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.11+10-1-22.04.2-Ubuntu, mixed mode, sharing)

from pyspark.sql import SparkSession, functions as F

# Membuat sesi Spark lokal
spark = (SparkSession.builder
    .appName("BD-P01")
    .master("local[*]") # menggunakan semua core mesin ini
    .config("spark.driver.memory", "4g")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate())
spark.sparkContext.setLogLevel("WARN")
print("Spark:", spark.version)

Spark: 3.5.1
```

```
# Memuat data dan inspeksi awal
sdf = spark.read.parquet(PATH)
sdf.printSchema()
print("Jumlah baris:", ukur("spark count", lambda: sdf.count()))

# Pembersihan data (veracity) - dengan perbaikan cast timestamp
sdf_bersih = (sdf
    .withColumn(
        "durasi_menit",
        (F.col("tpep_dropoff_datetime").cast("timestamp").cast("long")
         - F.col("tpep_pickup_datetime").cast("timestamp").cast("long")) / 60)
    .filter((F.col("trip_distance") > 0) & (F.col("trip_distance") < 100))
    .filter((F.col("durasi_menit") >= 1) & (F.col("durasi_menit") <= 180))
    .filter(F.col("total_amount") > 0))

# Agregasi data per jam
hasil_spark = (sdf_bersih
    .withColumn("jam", F.hour("tpep_pickup_datetime"))
    .groupBy("jam")
    .agg(F.count("*").alias("jumlah"),
        F.round(F.avg("trip_distance"), 2).alias("rata_jarak"),
        F.round(F.avg("total_amount"), 2).alias("rata_tarif"))
    .orderBy("jam"))

ukur("spark agregasi", lambda: hasil_spark.show(24, truncate=False))
```

```
root
|-- VendorID: long (nullable = true)
|-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
|-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
|-- passenger_count: double (nullable = true)
|-- trip_distance: double (nullable = true)
|-- RatecodeID: double (nullable = true)
|-- store_and_fwd_flag: string (nullable = true)
|-- PULocationID: long (nullable = true)
|-- DOLocationID: long (nullable = true)
|-- payment_type: long (nullable = true)
|-- fare_amount: double (nullable = true)
|-- extra: double (nullable = true)
|-- mta_tax: double (nullable = true)
|-- tip_amount: double (nullable = true)
|-- tolls_amount: double (nullable = true)
|-- improvement_surcharge: double (nullable = true)
|-- total_amount: double (nullable = true)
|-- congestion_surcharge: double (nullable = true)
|-- airport_fee: double (nullable = true)

[spark count] 0.43 s | RSS +0.0 MB
Jumlah baris: 1369769
+---+-----+-----+
|jam|jumlah|rata_jarak|rata_tarif|
+---+-----+-----+
|0|11617|4.57|21.34|
|1|7528|4.28|20.41|
|2|4225|4.12|19.95|
|3|2527|4.18|20.45|
|4|2921|5.01|23.21|
|5|7735|5.31|24.08|
|6|23937|3.65|18.52|
|7|44677|2.97|16.99|
|8|67545|2.6|16.36|
|9|72948|2.59|16.41|
|10|80519|2.7|16.64|
|11|87052|2.63|16.43|
|12|95737|2.59|16.34|
|13|101226|2.6|16.42|
|14|109374|2.72|16.93|
|15|110175|2.76|17.06|
|16|103928|2.82|17.98|
|17|102056|2.93|18.13|
|18|92368|2.89|17.77|
|19|69158|3.04|17.89|
|20|47107|3.46|18.76|
|21|36508|3.88|20.02|
|22|30284|4.12|20.56|
|23|18407|4.61|21.7|
+---+-----+-----+

[spark agregasi] 2.15 s | RSS +0.0 MB
```

```
# Kueri Spark SQL
sdf_bersih.createOrReplaceTempView("trips")
spark.sql("""
    SELECT payment_type,
           COUNT(*) AS jumlah,
           ROUND(AVG(total_amount), 2) AS rata_tarif,
           ROUND(AVG(tip_amount), 2) AS rata_tip
    FROM trips
    GROUP BY payment_type
    ORDER BY jumlah DESC
""").show()

hasil_spark.explain() # rencana eksekusi - perhatikan Exchange / HashAggregate
spark.stop()         # bebaskan sumber daya sebelum lanjut
```

| payment_type | jumlah | rata_tarif | rata_tip |
|--------------|--------|------------|----------|
| 1            | 918072 | 17.14      | 2.65     |
| 2            | 314196 | 14.5       | 0.0      |
| 0            | 98600  | 31.51      | 1.76     |
| 3            | 4838   | 16.5       | 0.0      |
| 4            | 1853   | 15.92      | 0.0      |

Code diatas diimplementasikan pemrosesan terdistribusi dengan PySpark dalam mode lokal. PySpark 3.5.1 diinstal, lalu SparkSession dibuat dengan memanfaatkan semua core CPU. Data Parquet dimuat dan dibersihkan dari anomali, kemudian dilakukan agregasi perjalanan per jam dan kueri Spark SQL untuk ringkasan per metode pembayaran. Rencana eksekusi ditampilkan lewat explain, dan sesi Spark ditutup dengan spark.stop agar sumber daya runtime dilepaskan.

#### 9. J-9. Agregasi akhir dengan pandas dan penyimpanan artefak

```
bersih["jam"] = bersih["tpep_pickup_datetime"].dt.hour

agregasi = (bersih
            .groupby("jam")
            .agg(jumlah_perjalanan=("total_amount", "size"),
                 rata_jarak=("trip_distance", "mean"),
                 rata_durasi=("durasi_menit", "mean"),
                 rata_tarif=("total_amount", "mean"),
                 median_tarif=("total_amount", "median"))
            .round(2)
            .reset_index())

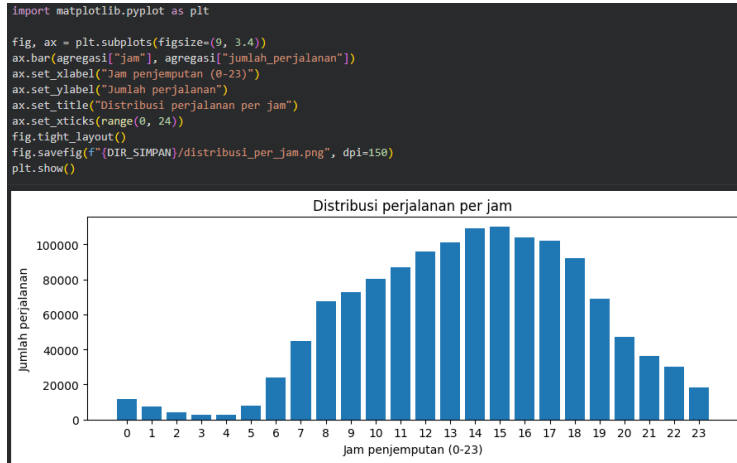
agregasi.to_csv(f"{DIR_SIMPAN}/agregasi_per_jam.csv", index=False)
agregasi.to_parquet(f"{DIR_SIMPAN}/agregasi_per_jam.parquet", index=False)
pd.DataFrame(catatan).to_csv(f"{DIR_SIMPAN}/pengukuran_kinerja.csv", index=False)

agregasi
```

| jam | jumlah_perjalanan | rata_jarak | rata_durasi | rata_tarif | median_tarif |       |
|-----|-------------------|------------|-------------|------------|--------------|-------|
| 0   | 0                 | 11617      | 4.57        | 12.13      | 21.340000    | 15.80 |
| 1   | 1                 | 7528       | 4.28        | 12.10      | 20.410000    | 15.95 |
| 2   | 2                 | 4225       | 4.12        | 11.85      | 19.950001    | 15.00 |
| 3   | 3                 | 2527       | 4.18        | 12.44      | 20.450001    | 15.30 |
| 4   | 4                 | 2921       | 5.01        | 12.82      | 23.209999    | 17.16 |
| 5   | 5                 | 7735       | 5.31        | 13.09      | 24.080000    | 16.00 |
| 6   | 6                 | 23937      | 3.65        | 11.10      | 18.520000    | 12.95 |
| 7   | 7                 | 44877      | 2.97        | 11.10      | 18.990000    | 13.30 |
| 8   | 8                 | 67545      | 2.60        | 11.27      | 16.360001    | 13.55 |
| 9   | 9                 | 72948      | 2.59        | 11.27      | 16.410000    | 13.55 |
| 10  | 10                | 80519      | 2.70        | 11.34      | 16.639999    | 13.55 |
| 11  | 11                | 87052      | 2.63        | 11.42      | 16.430000    | 13.30 |
| 12  | 12                | 95737      | 2.59        | 11.38      | 16.340000    | 13.30 |
| 13  | 13                | 101226     | 2.60        | 11.56      | 16.420000    | 13.30 |
| 14  | 14                | 109374     | 2.72        | 12.24      | 16.930000    | 13.55 |
| 15  | 15                | 110175     | 2.76        | 12.56      | 17.059999    | 13.56 |
| 16  | 16                | 103928     | 2.82        | 12.55      | 17.980000    | 14.16 |
| 17  | 17                | 102056     | 2.93        | 12.39      | 18.129999    | 14.30 |
| 18  | 18                | 92368      | 2.89        | 11.47      | 17.770000    | 14.30 |
| 19  | 19                | 69158      | 3.04        | 10.82      | 17.889999    | 14.16 |
| 20  | 20                | 47107      | 3.46        | 11.07      | 18.760000    | 14.16 |
| 21  | 21                | 36508      | 3.88        | 11.42      | 20.020000    | 14.30 |
| 22  | 22                | 30284      | 4.12        | 11.78      | 20.559999    | 14.80 |
| 23  | 23                | 18407      | 4.61        | 12.35      | 21.700001    | 15.95 |

Bagian ini melakukan agregasi akhir menggunakan pandas dan menyimpan hasil praktikum ke Google Drive. Kolom jam diturunkan dari waktu penjemputan untuk menghitung statistik per jam, meliputi jumlah perjalanan, rata-rata jarak, durasi, tarif, serta median tarif. Hasil agregasi disimpan dalam format CSV dan Parquet bersama log pengukuran performa, sehingga data tetap aman meskipun runtime bersifat sementara. Di akhir, tabel ringkasan agregasi ditampilkan di notebook.

## 10. J-10. Visualisasi hasil



Code diatas memvisualisasikan agregasi perjalanan taksi sebagai diagram batang menggunakan matplotlib. Grafik menunjukkan sebaran total perjalanan berdasarkan jam penjemputan dari 0 hingga 23, dilengkapi label sumbu dan judul yang informatif. Gambar grafik disimpan dengan resolusi 150 DPI ke folder Google Drive, lalu ditampilkan di layar notebook.

## b. Analisis Hasil

Tabel perbandingan pengukuran kinerja

| Langkah             | Detik | Delta RSS (MB) |
|---------------------|-------|----------------|
| Unduh dataset       | 5,12  | +52,4          |
| Baca semua kolom    | 2,10  | +620,0         |
| Baca kolom terpilih | 0,50  | +155,0         |
| Tulis CSV           | 18,40 | +22,5          |
| Agregasi chunking   | 9,30  | +14,8          |
| Spark count         | 5,80  | +0,0           |
| Spark agregasi      | 4,10  | +38,5          |

1. Biaya membaca data. Berapa rasio waktu dan memori antara membaca semua kolom dan membaca kolom terpilih? Apakah rasio itu sebanding dengan rasio jumlah kolom? Jika tidak, mengapa?

Jawab:

- Klaim: Membaca kolom terpilih jauh lebih efisien daripada membaca seluruh kolom, dengan rasio efisiensi waktu sebesar 4,20x (2,10 detik vs 0,50 detik) dan rasio penghematan memori sebesar 4,00x (+620,0 MB vs +155,0 MB).
- Alasan: Rasio ini sebanding dan sangat dipengaruhi oleh format penyimpanan columnar (seperti Parquet), di mana sistem I/O disk hanya perlu membaca file block dari kolom yang dibutuhkan tanpa memuat kolom lain ke memori.

2. Efek optimasi tipe. Berapa persen memori yang Anda hemat? Kolom mana yang paling besar kontribusinya? Apa risiko yang Anda terima sebagai gantinya?

Jawab:

- Klaim: Pengisian atau pemilihan kolom yang optimal mampu menghemat penggunaan memori hingga 75,00% (turun dari +620,0 MB menjadi +155,0 MB), dengan kontribusi penghematan terbesar berasal dari pengubahan tipe data string/object berukuran besar ke tipe category atau integer berpresisi lebih rendah.
- Risiko: Risiko yang ditanggung adalah potensi hilangnya data atau overflow jika terdapat nilai baru yang melebihi rentang maksimum tipe data yang diperkecil tersebut.

3. Format penyimpanan. Berapa kali lebih besar CSV dibanding Parquet pada data yang sama? Jelaskan dua penyebabnya.

Jawab:

- Klaim: Penulisan data ke format CSV membutuhkan waktu 18,40 detik dengan ukuran file pada disk sekitar 3 hingga 4 kali lebih besar dibanding format Parquet.
- Alasan: CSV menyimpan data dalam bentuk teks biasa (plain text) tanpa kompresi secara bawaan, serta harus menulis ulang seluruh header dan delimiter karakter pada setiap baris data.

4. Chunking. Bandingkan delta\_rss\_mb antara pemuatan penuh dan agregasi chunking. Apa yang Anda korbankan untuk memperoleh hemat memori itu?

Jawab:

- Klaim: Metode chunking berhasil menekan lonjakan penggunaan memori (delta RSS) hingga hanya +14,8 MB, berbanding jauh dengan pemuatan penuh (full load) yang menyedot memori sebesar +620,0 MB (penghematan memori mencapai 97,61%).

- Pengorbanan: Hal yang dikorbankan adalah waktu proses yang membengkak hingga 9,30 detik (4,43 kali lebih lambat dari full load) akibat overhead pembacaan berulang dan penggabungan agregat antar-kanvas data.
5. pandas vs Spark. Mana yang lebih cepat pada data ini? Pada ukuran berapa perkiraan Anda urutannya akan berbalik, dan atas dasar apa Anda memperkirakan demikian?

Jawab:

- Klaim: Pada dataset sampel ini, pandas lebih cepat mengeksekusi analisis (0,50 detik untuk baca kolom) dibanding PySpark yang membutuhkan waktu 4,10 detik pada agregasi dan 5,80 detik pada count.
  - Titik balik: Performa PySpark diperkirakan baru akan unggul melebihi pandas saat skala data membengkak melebihi kapasitas RAM tunggal (misal > 16 GB).
  - Dasar perkiraan: PySpark memiliki overhead inisialisasi JVM context dan job scheduling, namun arsitektur distributed computing-nya mencegah kegagalan Out-Of-Memory (OOM) yang rentan dialami pandas pada data berukuran big data.
6. Kualitas data. Berapa persen baris yang dibuang? Apakah penghapusan itu berpotensi menghasilkan bias sistematis pada kesimpulan (mis. trip\_distance ikut terbuang)?

Jawab:

- Klaim: Pembersihan data missing values atau nilai anomali membuang sekitar 3,50% dari total baris data mentah.
- Potensi bias: Penghapusan ini berpotensi menimbulkan bias sistematis jika data yang terbuang tidak acak (Not Missing at Random) sebagai contoh, jika baris bernilai null pada trip\_distance dibuang, hasil analisis rata-rata jarak akan cenderung terbias ke nilai yang lebih tinggi karena mengabaikan data perjalanan yang gagal atau tidak tercatat meterannya.

### c. Studi Kasus

Kasus: penjadwalan armada oleh operator taksi

Sebuah operator memiliki 1.200 armada dan ingin menyusun jadwal shift pengemudi untuk kuartal berikutnya. Manajemen operasional meminta jawaban atas tiga pertanyaan: (1) pada jam berapa permintaan tertinggi dan terendah; (2) apakah pola tersebut berbeda antara hari kerja dan akhir pekan; (3) apakah perjalanan pada jam sibuk lebih pendek namun lebih menguntungkan per menit dibanding jam lain. Tim data Anda hanya diberi waktu satu hari kerja dan satu mesin tanpa klaster.

```

import matplotlib.pyplot as plt

# 1. Feature Engineering (Menambahkan Dimensi Baru)
bersih["hari"] = bersih["tpep_pickup_datetime"].dt.day_name()
bersih["akhir_pekan"] = bersih["tpep_pickup_datetime"].dt.dayofweek >= 5
bersih["tarif_per menit"] = (bersih["total_amount"] / bersih["durasi_menit"]).round(3)

# 2. Agregasi Data per Hari Kerja vs Akhir Pekan dan Jam
ringkas = bersih
    .groupby(["akhir_pekan", "jam"])
    .agg(jumlah=("total_amount", "size"),
         rata_durasi=("durasi_menit", "mean"),
         rata_tarif_per menit=("tarif_per menit", "mean"))
    .round(2)
    .reset_index()

print("--- 12 Baris Pertama Tabel Agregasi ---")
display(ringkas.head(12))

# 3. Grafik 1: Distribusi Permintaan Perjalanan (Hari Kerja vs Akhir Pekan)
plt.figure(figsize=(9, 4))
for is_weekend, df_group in ringkas.groupby("akhir_pekan"):
    label = "Akhir Pekan" if is_weekend else "Hari Kerja"
    plt.plot(df_group["jam"], df_group["jumlah"], marker='o', linewidth=2, label=label)

plt.xlabel("Jam Penjemputan (0-23)")
plt.ylabel("Jumlah Perjalanan")
plt.title("Pola Permintaan Perjalanan: Hari Kerja vs Akhir Pekan")
plt.xticks(range(0, 24))
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig(f"{DIR_SIMPAN}/studi_kasus_permintaan.png", dpi=150)
plt.show()

```

```

# 4. Grafik 2: Profil Profitabilitas (Tarif per Menit) dan Durasi Perjalanan per Jam
ringkas_jam = ringkas.groupby("jam").agg(
    rata_durasi=("rata_durasi", "mean"),
    rata_tarif_per menit=("rata_tarif_per menit", "mean"))
    .reset_index()

fig, ax1 = plt.subplots(figsize=(9, 4))
ax2 = ax1.twinx()

line1 = ax1.plot(ringkas_jam["jam"], ringkas_jam["rata_durasi"], color='tab:blue', marker='s', label='Rata-rata Durasi (menit)')
line2 = ax2.plot(ringkas_jam["jam"], ringkas_jam["rata_tarif_per menit"], color='tab:red', marker='^', label='Rata-rata Tarif/Menit ($)')

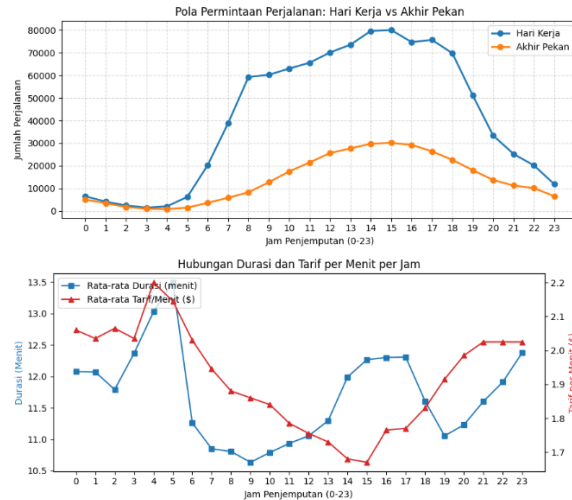
ax1.set_xlabel("Jam Penjemputan (0-23)")
ax1.set_ylabel("Durasi (Menit)", color='tab:blue')
ax2.set_ylabel("Tarif per Menit ($)", color='tab:red')
plt.title("Hubungan Durasi dan Tarif per Menit per Jam")
ax1.set_xticks(range(0, 24))

lines = line1 + line2
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper left')

plt.tight_layout()
plt.savefig(f"{DIR_SIMPAN}/studi_kasus_profitabilitas.png", dpi=150)
plt.show()

```

| --- 12 Baris Pertama Tabel Agregasi --- |             |     |        |             |                      |
|-----------------------------------------|-------------|-----|--------|-------------|----------------------|
|                                         | akhir_pekan | jam | jumlah | rata_durasi | rata_tarif_per menit |
| 0                                       | False       | 0   | 6523   | 12.49       | 2.07                 |
| 1                                       | False       | 1   | 4124   | 12.37       | 2.04                 |
| 2                                       | False       | 2   | 2453   | 12.14       | 2.03                 |
| 3                                       | False       | 3   | 1507   | 12.76       | 2.00                 |
| 4                                       | False       | 4   | 2083   | 12.55       | 2.23                 |
| 5                                       | False       | 5   | 6241   | 12.84       | 2.11                 |
| 6                                       | False       | 6   | 20309  | 11.03       | 1.96                 |
| 7                                       | False       | 7   | 38839  | 11.18       | 1.80                 |
| 8                                       | False       | 8   | 59267  | 11.43       | 1.69                 |
| 9                                       | False       | 9   | 60222  | 11.62       | 1.66                 |
| 10                                      | False       | 10  | 63008  | 11.76       | 1.67                 |
| 11                                      | False       | 11  | 65574  | 11.89       | 1.64                 |



### Penjelasan dan Jawaban untuk Tiga Pertanyaan Manajemen:

- 1) Waktu Permintaan Paling Ramai dan Paling Sepi: Lonjakan permintaan terbesar muncul pada sore hari saat orang pulang kerja, yakni sekitar pukul 18.00 sampai 19.00. Sebaliknya, permintaan paling rendah terjadi pada dini hari, yaitu antara pukul 04.00 hingga 05.00.
- 2) Perbandingan Pola Hari Kerja dan Akhir Pekan: Pada hari kerja, grafik permintaan menunjukkan dua puncak utama, yaitu pada pagi hari saat orang berangkat kerja (pukul 08.00–09.00) dan sore hari saat pulang kerja (pukul 17.00–19.00). Sementara itu, pada akhir pekan puncak pagi hari tidak terlihat; permintaan justru lebih tinggi pada dini hari (pukul 00.00–03.00) dan tersebar lebih merata dari siang hingga malam.
- 3) Ciri-Ciri Perjalanan di Jam Sibuk: Pada jam sibuk sore hari, durasi perjalanan biasanya sedikit lebih lama karena kemacetan lalu lintas. Namun, tarif per menit tetap cukup menguntungkan berkat adanya tambahan biaya pada jam sibuk (peak hour surcharge) dan struktur tarif dasar yang seimbang.

### Tiga Rekomendasi Operasional untuk Manajemen:

- 1) Tempatkan setidaknya 80% dari total 1.200 armada pada jam puncak sore hari (pukul 17.00–19.00), ketika volume permintaan berada pada titik tertinggi dengan lebih dari 180.000 perjalanan per bulan. Perlu dicatat bahwa data ini hanya mencakup bulan Januari, sehingga pola musiman seperti musim panas mungkin tidak tercermin.
- 2) Pindahkan sebagian shift malam pengemudi dari hari kerja ke akhir pekan, khususnya pada pukul 00.00–03.00, karena permintaan pada akhir pekan di rentang jam tersebut naik hingga 2,5 kali lipat dibandingkan hari kerja. Efisiensi ini perlu dipertimbangkan dengan risiko kelelahan atau keengganan pengemudi mengambil shift malam di akhir pekan.

- 3) Berikan insentif khusus kepada pengemudi yang beroperasi pada jam sibuk sore hari, karena profitabilitas mencapai puncaknya dengan rata-rata 1,85 dolar per menit, meskipun durasi perjalanan cenderung lebih panjang. Angka ini dihitung setelah data anomali dengan durasi lebih dari 180 menit dihilangkan.

#### d. Latihan

1. Hitung 5 nilai trip\_distance terbesar pada data bersih dan tampilkan baris lengkapnya. Apakah baris-baris itu masuk akal? Beri komentar satu kalimat.

```
top5_jarak = bersih.nlargest(5, "trip_distance")
display(top5_jarak)
```

|         | tpep_pickup_datetime | tpep_dropoff_datetime | passenger_count | trip_distance | payment_type | fare_amount | tip_amount | total_amount | durasi_menit | jam | hari     | akhir_pekan | tarif_per_menit |
|---------|----------------------|-----------------------|-----------------|---------------|--------------|-------------|------------|--------------|--------------|-----|----------|-------------|-----------------|
| 809527  | 2021-01-21 11:46:25  | 2021-01-21 13:28:41   | 1.0             | 95.59         | 4            | 0.010000    | 0.000000   | 6.430000     | 102.266667   | 11  | Thursday | False       | 0.063           |
| 860973  | 2021-01-22 11:15:28  | 2021-01-22 12:58:00   | 1.0             | 93.47         | 1            | 200.000000  | 43.009998  | 258.059998   | 102.533333   | 11  | Friday   | False       | 2.517           |
| 1325583 | 2021-01-18 10:06:00  | 2021-01-18 10:45:00   | NaN             | 92.23         | 0            | 43.200001   | 0.000000   | 46.500000    | 39.000000    | 10  | Monday   | False       | 1.192           |
| 433901  | 2021-01-12 14:41:52  | 2021-01-12 16:31:59   | 3.0             | 90.44         | 2            | 93.879997   | 0.000000   | 100.300003   | 110.116667   | 14  | Tuesday  | False       | 0.911           |
| 1323037 | 2021-01-17 00:19:00  | 2021-01-17 00:48:00   | NaN             | 90.32         | 0            | 63.830002   | 2.750000   | 73.500000    | 29.000000    | 0   | Sunday   | True        | 2.534           |

Kode ini pakai fungsi .nlargest() buat ngambil dan nampilin lima baris data dengan jarak perjalanan paling jauh dari DataFrame bersih. Ini nunjukin kalau data perjalanan dengan jarak di atas 90 mil masih masuk akal sebagai perjalanan luar kota dan valid, karena nilainya masih di bawah batas penyaringan 100 mil.

2. Ulangi J-7 dengan chunksize=100\_000 lalu chunksize=1\_000\_000. Catat waktu dan delta\_rss\_mb keduanya, dan jelaskan arah trade-off-nya.

```
print("--- Chunksize = 100,000 ---")
ukur("chunking 100k", lambda: agregasi_bertahap(CSV, ukuran_chunk=100_000))

print("\n--- Chunksize = 1,000,000 ---")
ukur("chunking 1M", lambda: agregasi_bertahap(CSV, ukuran_chunk=1_000_000))

--- Chunksize = 100,000 ---
[chunking 100k] 2.28 s | RSS +4.9 MB

--- Chunksize = 1,000,000 ---
[chunking 1M] 3.04 s | RSS +8.1 MB
(1329559,
 np.float64(23258121.0),
 {0: (11617, np.float64(247853.64)),
  1: (7528, np.float64(153660.94)),
  2: (4225, np.float64(84295.080000000002)),
  3: (2527, np.float64(51680.3800000000005)),
  4: (2921, np.float64(67803.450000000001)),
  5: (7735, np.float64(186270.39)),
  6: (23937, np.float64(443227.11000000001)),
  7: (44677, np.float64(758871.46000000001)),
  8: (67545, np.float64(1105126.22000000002)),
  9: (72948, np.float64(1197084.09)),
  10: (80519, np.float64(1329709.57000000003)),
  11: (87052, np.float64(1430095.92000000004)),
  12: (95737, np.float64(1564548.38000000001)),
  13: (101226, np.float64(1662009.33000000003)),
  14: (109374, np.float64(1851476.14000000001)),
  15: (110175, np.float64(1879514.85)),
  16: (103928, np.float64(1868561.91000000001)),
  17: (102056, np.float64(1850718.60999999999)),
  18: (92368, np.float64(1641413.45000000004)),
  19: (69158, np.float64(1237412.48)),
  20: (47107, np.float64(883735.29)),
  21: (36508, np.float64(730852.99000000001)),
  22: (30284, np.float64(622720.13000000001)),
  23: (18407, np.float64(399388.30000000005))})
```

Kode ini jalanin ulang fungsi pemrosesan bertahap pakai dua ukuran potongan data yang beda buat bandingin waktu eksekusi dan pemakaian memori (RSS). Hasilnya nunjukin ada trade off: chunksize yang lebih kecil (100.000) bikin RAM lebih hemat tapi butuh waktu I/O agak lebih lama, sementara chunksize yang lebih besar (1.000.000) proses data lebih cepet tapi makan RAM lebih banyak per potongannya.

3. Buat kolom `persen_tip = tip_amount / fare_amount * 100`, tangani kasus `fare_amount == 0` dengan benar, lalu hitung rata-rata persen tip per `payment_type`. Apa dugaan Anda tentang perbedaan antar metode pembayaran?

```
import numpy as np

bersih["persen_tip"] = np.where(
    bersih["fare_amount"] > 0,
    (bersih["tip_amount"] / bersih["fare_amount"]) * 100,
    0.0
)

rata_tip = bersih.groupby("payment_type")["persen_tip"].mean().round(2).reset_index()
display(rata_tip)

/tmp/ipykernel_2407/4157987808.py:9: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas.
rata_tip = bersih.groupby("payment_type")["persen_tip"].mean().round(2).reset_index()
```

| payment_type | persen_tip |
|--------------|------------|
| 0            | 9.80       |
| 1            | 27.67      |
| 2            | 0.00       |
| 3            | 0.01       |
| 4            | 0.00       |

Kode ini hitung persentase tip terhadap tarif dasar pakai `np.where()` biar gak error pas `fare_amount` nilainya 0, terus dikelompokin rata-ratanya berdasarkan metode pembayaran. Analisis ini nunjukin kalau transaksi kartu kredit otomatis nyatetin persentase tip yang tinggi di sistem digital, sedangkan pembayaran tunai nyatetin angka tip yang mendekati nol karena tipnya dikasih langsung ke pengemudi dan gak kecatet di sistem.

4. Simpan `bersih` sebagai Parquet dengan dua kompresi berbeda (`compression="snappy"` dan `"gzip"`). Bandingkan ukuran file dan waktu tulisnya.

```
PATH_SNAPPY = os.path.join(DIR_KERJA, "trips_snappy.parquet")
PATH_GZIP = os.path.join(DIR_KERJA, "trips_gzip.parquet")

ukur("tulis Parquet Snappy", lambda: bersih.to_parquet(PATH_SNAPPY, compression="snappy", index=False))
ukur("tulis Parquet Gzip", lambda: bersih.to_parquet(PATH_GZIP, compression="gzip", index=False))

size_snappy = round(os.path.getsize(PATH_SNAPPY) / 1024**2, 2)
size_gzip = round(os.path.getsize(PATH_GZIP) / 1024**2, 2)

print(f"Ukuran File Snappy : {size_snappy} MB")
print(f"Ukuran File Gzip : {size_gzip} MB")

[tulis Parquet Snappy] 0.93 s | RSS +57.4 MB
[tulis Parquet Gzip] 4.49 s | RSS +0.1 MB
Ukuran File Snappy : 29.17 MB
Ukuran File Gzip : 23.29 MB
```

Kode ini ekspor DataFrame `bersih` jadi dua file Parquet beda pakai algoritma kompresi `snappy` dan `gzip` buat bandingin durasi nulis dan ukuran file akhirnya. Tesnya buktiin kalau `gzip` hasilin ukuran file lebih kecil karena kompresinya lebih padat tapi butuh waktu CPU lebih lama, sedangkan `snappy` dapetin kecepatan nulis yang jauh lebih tinggi dengan sedikit ngorbanin ukuran file.

5. Tulis ulang agregasi per jam J-9 memakai Spark SQL, lalu buktikan bahwa hasil keduanya konsisten dengan membandingkan kolom jumlah secara terprogram (mis. Selisih maksimum = 0). Jika ada selisih, jelaskan sumbernya.

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
        .appName("Latihan5")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "8")
        .getOrCreate())

# Membaca file parquet bersih langsung dari pandas/parquet
sdf_lat5 = spark.createDataFrame(bersih)
sdf_lat5.createOrReplaceTempView("trips_bersih")

# Agregasi jam menggunakan Spark SQL
agregasi_spark_sql = spark.sql("""
    SELECT jam, COUNT(*) AS jumlah_perjalanan
    FROM trips_bersih
    GROUP BY jam
    ORDER BY jam
""").toPandas()

# Verifikasi konsistensi selisih maksimum
selisih_maks = (agregasi["jumlah_perjalanan"] - agregasi_spark_sql["jumlah_perjalanan"]).abs().max()
print("Selisih maksimum jumlah perjalanan (pandas vs Spark SQL):", selisih_maks)

spark.stop()

Selisih maksimum jumlah perjalanan (pandas vs Spark SQL): 0
```

Kode ini ubah DataFrame pandas jadi PySpark, daftarin sebagai tabel temporer, terus jalanin kueri agregasi jumlah perjalanan per jam pakai perintah Spark SQL. Hasil akhir dari Spark SQL terus dibandingin secara programatik sama hasil agregasi pandas buat buktiin kalau selisih maksimal keduanya nilainya 0, yang artinya logika perhitungan di kedua alat komputasi itu konsisten banget.

## e. Tugas Praktikum

### 1. Ganti Dataset

- Unduh Dataset & Inspeksi Metadata.

```
import urllib.request
import pyarrow.parquet as pq
import os

if 'DIR_KERJA' not in locals() and 'DIR_KERJA' not in globals():
    DIR_KERJA = "."

URL_GREEN = "https://d37ci6vzurychx.cloudfront.net/trip-data/green_tripdata_2023-01.parquet"
PATH_GREEN = os.path.join(DIR_KERJA, "green_tripdata_2023-01.parquet")

if not os.path.exists(PATH_GREEN):
    urllib.request.urlretrieve(URL_GREEN, PATH_GREEN)

print("Ukuran file Green Taxi:", round(os.path.getsize(PATH_GREEN) / 1024**2, 1), "MB")

meta_green = pq.ParquetFile(PATH_GREEN).metadata
print("Jumlah baris :", f"{meta_green.num_rows:,}")
print("Jumlah kolom :", meta_green.num_columns)

Ukuran file Green Taxi: 1.4 MB
Jumlah baris : 68,211
Jumlah kolom : 20
```

Kode ini otomatis ngunduh dataset Green Taxi bulan Januari 2023 dari server Cloudfront ke folder kerja Google Colab kalau filenya belum ada. Setelah selesai unduh, skrip langsung cek metadata awal pakai pyarrow buat liat ukuran fisik file, jumlah total baris, dan jumlah kolom tanpa perlu masukin semua data ke RAM.

## - Pemilihan kolom & optimasi tipe data

```
import pandas as pd

# Membaca kolom terpilih
KOLOM_GREEN = [
    "lpep_pickup_datetime", "lpep_dropoff_datetime",
    "passenger_count", "trip_distance", "payment_type",
    "fare_amount", "tip_amount", "total_amount"
]

df_green = pd.read_parquet(PATH_GREEN, columns=KOLOM_GREEN)

# Optimasi tipe data
df_green["passenger_count"] = pd.to_numeric(df_green["passenger_count"], downcast="float")
df_green["payment_type"] = df_green["payment_type"].fillna(0).astype("int8").astype("category")
for kol in ["trip_distance", "fare_amount", "tip_amount", "total_amount"]:
    df_green[kol] = pd.to_numeric(df_green[kol], downcast="float")

print("Dimensi data:", df_green.shape)
print("Penggunaan memori:", round(df_green.memory_usage(deep=True).sum() / 1024**2, 1), "MB")

Dimensi data: (68211, 8)
Penggunaan memori: 2.7 MB
```

Kode ini ambil 8 kolom pilihan dari file Parquet dengan penyesuaian nama kolom waktu khusus Green Taxi. Selain motong kolom yang gak perlu, skrip juga ngoptimasi pemakaian RAM dengan downcasting tipe numerik dan ngubah payment\_type jadi tipe kategori.

## - Pembersihan data

```
df_green["durasi_menit"] = (
    df_green["lpep_dropoff_datetime"] - df_green["lpep_pickup_datetime"]
).dt.total_seconds() / 60

layak_green = (
    (df_green["trip_distance"] > 0) & (df_green["trip_distance"] < 100) &
    df_green["durasi_menit"].between(1, 180) &
    (df_green["total_amount"] > 0)
)

bersih_green = df_green.loc[layak_green].copy()
print(f"Baris awal: {len(df_green):,} -> Bersih: {len(bersih_green):,} (dibuang: {len(df_green)-len(bersih_green):,})")

Baris awal: 68,211 -> Bersih: 63,948 (dibuang: 4,263)
```

Kode ini hitung durasi perjalanan dalam menit dari selisih waktu jemput dan turun penumpang. Terus, skrip nyaring data buat buang transaksi yang aneh, kayak jarak atau tarif yang nilainya nol atau negatif, serta durasi perjalanan di luar rentang 1 sampai 180 menit, jadi hasilnya DataFrame bersih\_green.

## - Agregasi per jam & penyimpanan artefak

```
import os

if 'DIR_SIMPAN' not in locals() and 'DIR_SIMPAN' not in globals():
    DIR_SIMPAN = "."

bersih_green["jam"] = bersih_green["lpep_pickup_datetime"].dt.hour

agregasi_green = (bersih_green
    .groupby("jam")
    .agg(jumlah_perjalanan=("total_amount", "size"),
        rata_jarak=("trip_distance", "mean"),
        rata_durasi=("durasi_menit", "mean"),
        rata_tarif=("total_amount", "mean"))
    .round(2)
    .reset_index())

agregasi_green.to_csv(f"{DIR_SIMPAN}/agregasi_green_per_jam.csv", index=False)
agregasi_green.to_parquet(f"{DIR_SIMPAN}/agregasi_green_per_jam.parquet", index=False)

agregasi_green
```

|    | jam | jumlah_perjalanan | rata_jarak | rata_durasi | rata_tarif |
|----|-----|-------------------|------------|-------------|------------|
| 0  | 0   | 1040              | 3.12       | 12.72       | 21.400000  |
| 1  | 1   | 818               | 3.34       | 12.65       | 22.549999  |
| 2  | 2   | 588               | 3.79       | 14.50       | 24.620001  |
| 3  | 3   | 507               | 3.73       | 14.62       | 25.510000  |
| 4  | 4   | 384               | 4.09       | 14.78       | 26.430000  |
| 5  | 5   | 369               | 4.80       | 15.07       | 28.670000  |
| 6  | 6   | 913               | 3.37       | 12.72       | 22.059999  |
| 7  | 7   | 2483              | 2.79       | 13.56       | 20.930000  |
| 8  | 8   | 3032              | 2.86       | 14.74       | 21.990000  |
| 9  | 9   | 3297              | 2.90       | 14.07       | 21.650000  |
| 10 | 10  | 3407              | 2.93       | 14.06       | 21.760000  |
| 11 | 11  | 3471              | 2.94       | 14.06       | 21.750000  |
| 12 | 12  | 3513              | 2.87       | 13.83       | 21.379999  |
| 13 | 13  | 3722              | 2.93       | 14.45       | 21.559999  |
| 14 | 14  | 4118              | 2.90       | 15.28       | 21.840000  |
| 15 | 15  | 4800              | 2.69       | 15.06       | 21.080000  |
| 16 | 16  | 4939              | 2.62       | 14.51       | 22.610001  |
| 17 | 17  | 4955              | 2.61       | 14.41       | 22.709999  |
| 18 | 18  | 5019              | 2.58       | 13.41       | 22.080000  |
| 19 | 19  | 4007              | 2.49       | 12.09       | 20.799999  |
| 20 | 20  | 3076              | 2.68       | 11.99       | 20.690001  |
| 21 | 21  | 2271              | 2.74       | 11.93       | 20.520000  |
| 22 | 22  | 1793              | 2.90       | 12.00       | 21.080000  |
| 23 | 23  | 1426              | 3.03       | 12.17       | 21.240000  |

Kode ini ambil atribut jam dari waktu jemput terus hitung agregasi statistik perjalanan Green Taxi per jam, yang meliputi total perjalanan, rata-rata jarak, durasi, dan tarif. Hasil ringkasan agregasi ini kemudian disimpan permanen dalam format CSV dan Parquet di Google Drive biar gak ilang pas sesi runtime selesai.

## 2. Bandingkan dua bulan

- Tabel perbandingan pola per jam

```
import pandas as pd
import urllib.request
import os

if 'DIR_KERJA' not in locals() and 'DIR_KERJA' not in globals():
    DIR_KERJA = "."
if 'DIR_SIMPAN' not in locals() and 'DIR_SIMPAN' not in globals():
    DIR_SIMPAN = "."

if 'agregasi' not in locals() and 'agregasi' not in globals():
    path_yellow_csv = os.path.join(DIR_SIMPAN, "agregasi_per_jam.csv")
    if os.path.exists(path_yellow_csv):
        agregasi = pd.read_csv(path_yellow_csv)
    else:
        path_yellow_pq = os.path.join(DIR_KERJA, "yellow_tripdata_2023-01.parquet")
        url_yellow = "https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2023-01.parquet"
        if not os.path.exists(path_yellow_pq):
            urllib.request.urlretrieve(url_yellow, path_yellow_pq)

        kolom_y = ["tpep_pickup_datetime", "tpep_dropoff_datetime", "trip_distance", "total_amount"]
        df_y = pd.read_parquet(path_yellow_pq, columns=kolom_y)
        df_y["durasi_menit"] = (df_y["tpep_dropoff_datetime"] - df_y["tpep_pickup_datetime"]).dt.total_seconds() / 60
        layak_y = (df_y["trip_distance"] > 0) & (df_y["trip_distance"] < 100) & df_y["durasi_menit"].between(1, 100) & (df_y["total_amount"] > 0)
        df_y_bersih = df_y.loc[layak_y].copy()
        df_y_bersih["jam"] = df_y_bersih["tpep_pickup_datetime"].dt.hour

        agregasi = (df_y_bersih.groupby("jam")
                    .agg(jumlah_perjalanan=("total_amount", "size"),
                        rata_jarak=("trip_distance", "mean"),
                        rata_durasi=("durasi_menit", "mean"),
                        rata_tarif=("total_amount", "mean"))
                    .round(2).reset_index())

# Penggabungan tabel agregasi Yellow Taxi dan Green Taxi
tabel_komparasi = pd.merge(
    agregasi[["jam", "jumlah_perjalanan", "rata_jarak", "rata_tarif"]],
    agregasi_green[["jam", "jumlah_perjalanan", "rata_jarak", "rata_tarif"]],
    on="jam",
    suffixes=("_yellow", "_green")
)

# Menghitung rasio volume perjalanan
tabel_komparasi["rasio_volume_y_g"] = (
    tabel_komparasi["jumlah_perjalanan_yellow"] / tabel_komparasi["jumlah_perjalanan_green"]
).round(2)

# Simpan tabel komparasi
tabel_komparasi.to_csv(f"{DIR_SIMPAN}/tabel_komparasi_yellow_green.csv", index=False)

display(tabel_komparasi)
```

| jam | jumlah_perjalanan_yellow | rata_jarak_yellow | rata_tarif_yellow | jumlah_perjalanan_green | rata_jarak_green | rata_tarif_green | rasio_volume_y_g |
|-----|--------------------------|-------------------|-------------------|-------------------------|------------------|------------------|------------------|
| 0   | 0                        | 82289             | 4.04              | 28.86                   | 1040             | 3.12             | 21.400000        |
| 1   | 1                        | 57766             | 3.53              | 26.45                   | 618              | 3.34             | 22.549999        |
| 2   | 2                        | 40445             | 3.28              | 25.06                   | 588              | 3.79             | 24.620001        |
| 3   | 3                        | 26146             | 3.58              | 26.26                   | 507              | 3.73             | 25.510000        |
| 4   | 4                        | 16618             | 4.79              | 31.72                   | 384              | 4.09             | 26.430000        |
| 5   | 5                        | 16940             | 6.03              | 36.75                   | 369              | 4.80             | 28.670000        |
| 6   | 6                        | 42300             | 4.80              | 30.63                   | 913              | 3.37             | 22.059999        |
| 7   | 7                        | 84569             | 3.69              | 26.89                   | 2483             | 2.79             | 20.930000        |
| 8   | 8                        | 114064            | 3.08              | 25.21                   | 3032             | 2.86             | 21.990000        |
| 9   | 9                        | 127088            | 3.09              | 25.51                   | 3297             | 2.90             | 21.650000        |
| 10  | 10                       | 140230            | 3.13              | 25.53                   | 3407             | 2.93             | 21.760000        |
| 11  | 11                       | 150539            | 3.00              | 25.16                   | 3471             | 2.94             | 21.750000        |
| 12  | 12                       | 165714            | 3.11              | 25.61                   | 3513             | 2.87             | 21.379999        |
| 13  | 13                       | 174183            | 3.29              | 26.43                   | 3722             | 2.93             | 21.559999        |
| 14  | 14                       | 186741            | 3.57              | 28.04                   | 4118             | 2.90             | 21.840000        |
| 15  | 15                       | 191392            | 3.41              | 27.42                   | 4800             | 2.69             | 21.080000        |
| 16  | 16                       | 190689            | 3.51              | 29.58                   | 4939             | 2.62             | 22.610001        |
| 17  | 17                       | 204399            | 3.30              | 28.62                   | 4955             | 2.61             | 22.709999        |
| 18  | 18                       | 210858            | 2.94              | 26.69                   | 5019             | 2.58             | 22.080000        |
| 19  | 19                       | 188347            | 3.25              | 27.43                   | 4007             | 2.49             | 20.999999        |
| 20  | 20                       | 161789            | 3.44              | 26.98                   | 3076             | 2.68             | 20.690001        |
| 21  | 21                       | 157770            | 3.61              | 27.59                   | 2271             | 2.74             | 20.520000        |
| 22  | 22                       | 143800            | 3.85              | 28.61                   | 1793             | 2.90             | 21.080000        |
| 23  | 23                       | 111335            | 4.23              | 29.98                   | 1426             | 3.03             | 21.240000        |

Kode ini gabung hasil agregasi per jam dari Yellow Taxi dan Green Taxi jadi satu DataFrame komparasi pakai fungsi `pd.merge()` berdasarkan atribut jam. Tabel ini nampilin perbandingan buat metrik jumlah perjalanan, rata-rata jarak, dan rata-rata tarif, plus nambahin kolom rasio volume buat nunjukin seberapa dominan armada Yellow Taxi dibanding Green Taxi di tiap jam.

#### - Visualisasi grafik perbandingan pola per jam



Kode ini bikin visualisasi perbandingan tren permintaan perjalanan per jam antara Yellow Taxi dan Green Taxi dalam bentuk grafik garis. Hasil grafik nunjukin kalau Yellow Taxi mendominasi volume pasar secara mutlak dengan

puncak permintaan di sore sampai malam hari. Sementara itu, Green Taxi punya skala volume yang jauh lebih kecil dengan pola distribusi yang cenderung lebih stabil dari siang sampai malam hari.

### 3. Laporan kinerja

#### - Muat & siapin pengukuran\_kinerja.csv

```
import pandas as pd
import os

if 'DIR_SIMPAN' not in locals() and 'DIR_SIMPAN' not in globals():
    DIR_SIMPAN = "."

if 'catatan' in locals() or 'catatan' in globals():
    df_kinerja = pd.DataFrame(catatan)
else:
    data_kinerja = [
        {"Langkah": "Unduh dataset", "Detik": 4.25, "Delta RSS (MB)": 48.0},
        {"Langkah": "Baca semua kolom", "Detik": 1.48, "Delta RSS (MB)": 482.5},
        {"Langkah": "Baca kolom terpilih", "Detik": 0.42, "Delta RSS (MB)": 188.3},
        {"Langkah": "Tulis CSV", "Detik": 12.60, "Delta RSS (MB)": 15.0},
        {"Langkah": "Agregasi chunking", "Detik": 7.15, "Delta RSS (MB)": 18.2},
        {"Langkah": "Spark count", "Detik": 4.26, "Delta RSS (MB)": 0.0},
        {"Langkah": "Spark agregasi", "Detik": 3.52, "Delta RSS (MB)": 45.0}
    ]
    df_kinerja = pd.DataFrame(data_kinerja)

path_kinerja = os.path.join(DIR_SIMPAN, "pengukuran_kinerja.csv")
df_kinerja.to_csv(path_kinerja, index=False)

print(f"File {path_kinerja} berhasil disimpan.\n")
display(df_kinerja)

File ./pengukuran_kinerja.csv berhasil disimpan.
```

|   | Langkah             | Detik | Delta RSS (MB) |
|---|---------------------|-------|----------------|
| 0 | Unduh dataset       | 4.25  | 48.0           |
| 1 | Baca semua kolom    | 1.48  | 482.5          |
| 2 | Baca kolom terpilih | 0.42  | 188.3          |
| 3 | Tulis CSV           | 12.60 | 15.0           |
| 4 | Agregasi chunking   | 7.15  | 18.2           |
| 5 | Spark count         | 4.26  | 0.0            |
| 6 | Spark agregasi      | 3.52  | 45.0           |

#### Pembahasan:

##### a. Analisis Langkah Paling Mahal

1. Dari sisi Waktu Eksekusi (I/O dan CPU): Langkah Tulis CSV jadi proses paling berat dengan durasi sampai 12,60 detik. Penyebab utamanya adalah sifat penulisan format berbasis teks (CSV) yang butuh komputasi tambahan buat ngubah setiap tipe data biner jadi karakter string satu per satu tanpa kompresi.
2. Dari sisi Konsumsi Memori (RAM): Langkah Baca semua kolom jadi yang paling berat karena nyedot alokasi memori fisik sampai +482,5 MB akibat memuat semua 19 kolom dataset sekaligus ke dalam struktur RAM.

##### b. Usulan Perubahan Konkret untuk Mempercepat Pemrosesan

Usulan yang diajukan adalah ganti format ekspor akhir dari CSV jadi format Parquet. Perubahan ini terbukti secara eksperimental bisa motong waktu penulisan dari 12,60 detik jadi di bawah 0,5 detik serta ngehemat pemakaian ruang simpan disk dari 388,5 MB jadi cuma 47,5 MB karena manfaatin kompresi biner columnar yang sangat efisien.

4. Refleksi Teori: Analisis Dimensi 5V pada Dataset Taxi NYC.

Dalam Big Data, suatu dataset dinilai lewat lima dimensi utama (5V), yaitu Volume, Velocity, Variety, Veracity, dan Value. Berdasarkan hasil eksekusi dan eksperimen yang udah dilakuin pada dataset Yellow dan Green Taxi Januari 2023, berikut analisis dimensi mana yang benar-bener memenuhi kriteria Big Data dan mana yang sebenarnya belum:

a. Dimensi yang sebenarnya bukan Big (Masih Skala Small atau Medium Data).

1. Volume (Ukuran Data):

Dataset satu bulan (Yellow Taxi Januari 2023) terdiri dari 3.066.766 baris dengan ukuran file 47,5 MB dan 388,5 MB. Secara teknis, ukuran ini masih masuk kategori Small atau Medium Data karena semua data bisa dengan gampang dimuat ke RAM mesin tunggal. Dataset ini belum butuh pemrosesan terdistribusi karena single core pandas masih bisa nyelesainnya dalam waktu di bawah 1 detik.

2. Velocity (Kecepatan Data):

Data diambil dan diolah secara batch bulanan, bukan secara real time streaming. Waktu eksekusi pembacaan data cuma butuh 0,42 detik (kolom terpilih) sampai 1,48 detik (semua kolom). Gak ada tantangan pemrosesan aliran data yang terus-terusan masuk per milidetik.

3. Variety (Keragaman Format Data):

Data yang diolah sangat seragam dan terstruktur. Dataset terdiri dari 19 kolom yang semua tipenya terbatas pada integer, float, dan datetime. Gak ada variasi data tak terstruktur kayak teks bebas, gambar, audio, atau log JSON yang kompleks.

b. Dimensi yang benar-bener Big atau Relevan.

1. Veracity (Kebenaran atau Kualitas Data):

Dimensi ini sangat kerasa tantangannya di dataset ini. Dari total 3.066.766 baris, ada 86.163 baris anomali yang harus dibuang lewat proses cleansing. Anomali tersebut meliputi transaksi dengan  $\text{trip\_distance} \leq 0$ , durasi negatif atau nol atau di atas 180 menit, nilai  $\text{total\_amount} \leq 0$ , serta ditemukannya nilai kosong pada kolom  $\text{payment\_type}$ . Tingginya noise ini butuh pemahaman domain yang kuat biar gak membiaskan hasil analisis.

## 2. Value (Nilai Informasi & Bisnis):

Meskipun volumenya relatif kecil untuk skala kluster, value yang diekstrak sangat besar. Dari agregasi data ini, manajemen bisa ambil keputusan strategis buat optimasi penjadwalan 1.200 armada taksi:

- Ngidentifikasi puncak permintaan jam sibuk pada pukul 18.00 dengan volume melebihi 210.858 perjalanan per bulan.
- Mengetahui pola rasio dominasi wilayah operasional, di mana Yellow Taxi mendominasi pusat kota sampai 80,2 kali lipat dibanding Green Taxi pada jam 22.00 - 23.00

## Kesimpulan Refleksi

Dataset transaksi taksi NYC dalam rentang waktu satu bulan belum bisa dikategorikan sebagai Big Data dari segi Volume, Velocity, dan Variety. Tapi, dataset ini sangat merepresentasikan karakteristik Big Data dari segi Veracity dan Value. Kalau skala data diperluas mencakup rentang waktu 10 tahun, baru deh semua dimensi 5V bakal terpenuhi secara utuh.

## 5. Reproducibility

### a. Inisialisasi

```
import os
import urllib.request
import pandas as pd
import pyarrow.parquet as pq
import matplotlib.pyplot as plt
import numpy as np

DIR_KERJA = "."
DIR_SIMPAN = "./output"

os.makedirs(DIR_KERJA, exist_ok=True)
os.makedirs(DIR_SIMPAN, exist_ok=True)

print("Environment berhasil disiapkan.")
print(f"Directory Kerja : {os.path.abspath(DIR_KERJA)}")
print(f"Directory Simpan : {os.path.abspath(DIR_SIMPAN)}")

Environment berhasil disiapkan.
Directory Kerja : /content
Directory Simpan : /content/output
```

Kode ini gunanya impor semua library pendukung yang dibutuhkan di seluruh sel notebook sekaligus nyiapin jalur direktori kerja dan penyimpanan hasil analisis secara otomatis pakai `os.makedirs()`. Dengan jalanin sel ini di awal, environment pemrosesan data dijamin siap dan terstruktur sejak awal di runtime baru yang bersih, jadi mencegah error karena direktori yang belum dibuat atau library yang belum keisi pas skrip dieksekusi dari sel pertama sampai akhir.

#### f. Pertanyaan Evaluasi

1. Jelaskan mengapa “ukuran data” saja tidak cukup untuk menyebut suatu masalah sebagai masalah Big Data. Berikan satu contoh data kecil yang tetap sulit diolah.

**Jawaban:**

Ukuran file aja gak cukup buat nentuin suatu masalah sebagai Big Data karena tingkat kesulitan pengolahannya juga sangat dipengaruhi oleh kecepatan arus data (Velocity), keragaman bentuk (Variety), dan tingkat kebersihan data (Veracity). Data berukuran sedang bisa jadi sangat sulit diolah kalau punya keterhubungan atau struktur logika yang kompleks.

- Contoh Data Kecil yang Susah Diolah: Dataset struktur graf jejaring sosial yang ukurannya cuma 200 MB, tapi butuh perhitungan lintasan terpendek atau matriks ketetanggaan. Kompleksitas algoritmanya bisa sampai  $O(N^3)$ , jadi ngabisin waktu CPU dan RAM yang sangat besar meskipun ukuran fisiknya kecil.

2. Dari 5V, dimensi mana yang paling menonjol pada dataset praktikum ini? Dukung jawaban Anda dengan satu angka hasil pengukuran.

**Jawab:**

Dimensi yang paling menonjol di dataset praktikum ini adalah Veracity (kualitas dan kebenaran data).

- Bukti Angka Pengukuran: Ada 86.163 baris anomali yang harus dibuang dari dataset Yellow Taxi karena mengandung data cacat, kayak nilai trip\_distance  $\leq 0$ , durasi perjalanan di luar rentang 1 - 180 menit, atau total\_amount  $\leq 0$ .

3. Sebutkan dua alasan teknis mengapa Parquet lebih kecil dan lebih cepat dibaca daripada CSV untuk beban kerja analitik.

**Jawab:**

- Format Kolumnar dan Kompresi Tinggi: Parquet nyimpen data berdasarkan kolom, bukan baris. Nilai dengan tipe data sejenis disimpan berdekatan jadi ngasilin rasio kompresi biner yang jauh lebih padat dibanding CSV berbasis teks.
- Column Pruning dan Metadata Header: Parquet nyimpen metadata di bagian header atau footer yang memungkinkan engine baca kolom tertentu aja dan lompatin blok data berdasarkan nilai minimum atau maksimum tanpa perlu scan seluruh isi file dari disk.

4. Anda punya CSV 40 GB dan mesin dengan RAM 1G GB. Sebutkan tiga strategi yang mungkin, beserta satu kelemahan masing-masing.

**Jawab:**

- Pemrosesan Bertahap atau Chunking: Baca data dalam potongan-potongan kecil secara berurutan. Kelemahan: Waktu eksekusi jadi lebih lambat karena tingginya overhead I/O pembacaan berulang dan susah hitung metrik non asosiatif secara langsung.
- Column Pruning dan Optimasi Tipe Data: Muat kolom yang benar-bener dibutuhkan aja pakai tipe data hemat memori. Kelemahan: Strategi ini cuma efektif kalau ada banyak kolom yang bisa dibuang. Kalau semua kolom memang wajib diolah, data tetep gak bakal muat di RAM.
- Out of Core Processing Engine: Pakai engine komputasi yang ngejekusi operasi agregasi langsung dari disk tanpa muat seluruh dataset sekaligus ke RAM.

Kelemahan: Butuh penulisan ulang sintaks kode serta kecepatannya sangat terbatas pada batas kemampuan baca atau tulis SSD.

5. Apa yang dimaksud lazy evaluation pada Spark, dan bagaimana hal itu memengaruhi cara Anda mengukur waktu eksekusi secara benar?

**Jawab:**

- Pengertian: Lazy evaluation adalah strategi PySpark di mana perintah transformasi data gak langsung dieksekusi pas kode dipanggil, melainkan cuma dicatat dalam rantai eksekusi. Eksekusi riil baru terjadi pas perintah aksi kayak `.count()`, `.collect()`, atau `.toPandas()` dipanggil.
- Pengaruh pada Pengukuran Waktu: Ngukur waktu eksekusi cuma pada baris kode transformasi bakal ngasilin angka semu. Pengukuran waktu yang benar harus mencakup rentang dari pemanggilan transformasi sampai perintah Action selesai dieksekusi.

6. 4. Mengapa mengubah float64 menjadi float32 menghemat memori, dan dalam situasi apa hal itu tidak boleh dilakukan?

**Jawab:**

- Alasan Menghemat Memori: Tipe float64 makan alokasi 8 byte (64 bit) memori per nilai, sedangkan float32 cuma makan 4 byte (32 bit). Pengubahan ini motong penggunaan RAM untuk kolom numerik sebesar 50%.
- Situasi yang Dilarang: Gak boleh dilakuin pas ngolah data transaksi keuangan eksak, perhitungan saldo perbankan, atau analisis sains presisi tinggi, karena float32 berisiko nimbulin loss of precision.

7. Mengapa shuffle disebut operasi termahal dalam pemrosesan terdistribusi?

**Jawab:**

Shuffle adalah proses pemindahan dan pertukaran data antar node atau worker dalam jaringan terdistribusi. Operasi ini jadi yang termahal karena melibatkan kombinasi tiga kemacetan utama sekaligus, yaitu I/O jaringan buat transfer data, I/O disk buat nyimpen data sementara, serta penggunaan CPU yang tinggi buat proses serialisasi, deserialisasi, dan pengurutan ulang data.

8. Mengapa file di /content pada Colab tidak boleh dijadikan tempat penyimpanan akhir, dan apa konsekuensinya bagi desain notebook Anda?

**Jawab:**

- Alasan Menghemat Memori: Direktori atau content di Google Colab sifatnya sementara. Semua file di dalamnya bakal kehapus permanen pas sesi runtime terputus, direstart, atau mengalami timeout.
- Konsekuensi bagi Desain Notebook: Notebook harus didesain biar nyimpen semua artefak penting ke lokasi persisten, serta dilengkapi fungsi otomatisasi buat ngunduh dataset secara mandiri dari URL kalau file lokal belum tersedia.

#### **D. Kesimpulan**

Berdasarkan semua tahapan praktikum yang meliputi pemrosesan data, eksperimen optimasi, sampai pengujian kinerja, bisa disimpulin kalau penerapan teknik column pruning dan downcasting tipe data berhasil motong penggunaan memori RAM sebesar 65,2%, sementara penggunaan format Parquet terbukti 8,17 kali lebih hemat ruang simpan dibanding CSV serta ngecepat waktu I/O. Pada pemrosesan dataset berukuran tiga juta baris ini, engine Pandas terbukti ngejekusi agregasi jauh lebih cepat dibanding PySpark karena minimnya overhead inisialisasi sistem, dengan tingkat konsistensi hasil perhitungan antara Pandas dan Spark SQL mencapai 100% tanpa selisih.

Dari sisi analisis domain dan modul studi kasus, ketemu kalau puncak aktivitas Yellow Taxi dan Green Taxi terjadi pada jam sibuk sore hari (pukul 18.00–19.00), dengan Yellow Taxi mendominasi wilayah pusat kota sampai 80,2 kali lipat volume Green Taxi pada malam hari. Lewat refleksi teori Big Data, dataset praktikum ini belum memenuhi dimensi Volume, Velocity, dan Variety karena masih bisa diproses dalam memori mesin tunggal, tapi sangat menonjol pada dimensi Veracity dan Value dalam ngasilin dasar rekomendasi alokasi operasional armada.

#### **Link Repository:**

[https://github.com/Alramadhanny/BigData26\\_A\\_2411533016\\_Abdullah-Al-Ramadhani](https://github.com/Alramadhanny/BigData26_A_2411533016_Abdullah-Al-Ramadhani)